

Lab 7: In-Class Task

Python — Fundamentals & OOP

Duration: 30 minutes

Task: Build a Student Grade Manager

Create a Python program that manages student records, calculates grades, and demonstrates OOP principles with classes and inheritance.

Requirements

Part 1: Student Class (10 minutes)

Create a `Student` class with the following:

Attributes (in `__init__`):

- `name` — student's full name (string)
- `student_id` — unique identifier (string)
- `grades` — list of numeric grades (list of floats/ints)

Methods:

- `add_grade(grade)` — adds a grade to the grades list
 - `get_average()` — returns the average of all grades (return 0 if no grades)
 - `get_letter_grade()` — returns letter grade based on average:
 - 90-100: "A"
 - 80-89: "B"
 - 70-79: "C"
 - 60-69: "D"
 - Below 60: "F"
 - `__str__` — returns formatted string: "Name (ID) - Average: X.X (Letter)"
-

Part 2: Child Classes (10 minutes)

Create two classes that inherit from `Student`:

GraduateStudent class:

- Additional attribute: `thesis_title` (string)
- Additional method: `get_status()` — returns "Graduate Student"
- Override `__str__` to include thesis title

UndergraduateStudent class:

- Additional attribute: `year` (integer: 1, 2, 3, or 4)
 - Additional method: `get_status()` — returns "Undergraduate - Year X"
 - Override `__str__` to include year
-

Part 3: Grade Manager Functions (5 minutes)

Create the following standalone functions:

`calculate_class_average(students)`

- Takes a list of Student objects
- Returns the average of all students' averages

`get_top_student(students)`

- Takes a list of Student objects
- Returns the student with the highest average

`get_students_by_grade(students, letter)`

- Takes a list of students and a letter grade
 - Returns a list of students who have that letter grade
-

Part 4: Main Program (5 minutes)

Create a `main.py` file that:

1. Creates at least 5 student objects:
 - 2 GraduateStudent instances
 - 3 UndergraduateStudent instances
2. Adds multiple grades to each student (at least 3 grades per student)
3. Stores all students in a single list
4. Demonstrates the following:

- Print all students using a loop
 - Print the class average
 - Print the top student
 - Print all students with grade "A"
 - Call `get_status()` on each student (polymorphism)
-

Expected Output Example

```
=== All Students ===
John Smith (G001) - Average: 92.5 (A) - Thesis: Machine Learning in Healthcare
Jane Doe (G002) - Average: 88.0 (B) - Thesis: Data Privacy Laws
Alex Johnson (U001) - Average: 75.3 (C) - Year: 2
Maria Garcia (U002) - Average: 91.0 (A) - Year: 3
Tom Wilson (U003) - Average: 68.5 (D) - Year: 1

=== Class Statistics ===
Class Average: 83.06
Top Student: John Smith with 92.5

=== Students with Grade A ===
John Smith
Maria Garcia

=== Student Status ===
John Smith: Graduate Student
Jane Doe: Graduate Student
Alex Johnson: Undergraduate - Year 2
Maria Garcia: Undergraduate - Year 3
Tom Wilson: Undergraduate - Year 1
```

File Structure

```
inclass/
  student.py      (Student base class)
  graduate.py     (GraduateStudent class)
  undergraduate.py (UndergraduateStudent class)
  utils.py        (standalone functions)
  main.py         (main program)
```

Or combine classes in one file:

```
inclass/  
    models.py      (all classes)  
    main.py        (main program with functions)
```

Python Concepts Being Tested

- **Classes and `__init__`** — constructor with attributes
 - **Instance methods** — operating on object data
 - **`__str__` method** — string representation
 - **Inheritance** — child classes extending parent
 - **Method overriding** — customizing behavior in child classes
 - **Polymorphism** — calling same method on different types
 - **Lists** — storing objects, iteration
 - **Functions** — parameters, return values
 - **Conditionals** — if/elif/else for letter grades
 - **Loops** — iterating over students
 - **Modules and imports** — organizing code in files
-

Grading Criteria

- Student class with all required attributes and methods
- Two child classes with proper inheritance
- Method overriding implemented correctly
- Standalone functions working with student list
- Main program demonstrating all features
- Code follows PEP 8 style guidelines
- Program runs without errors

Good luck!